END-SEMESTER REVISION · 2026

HCI Study Guide

Guidelines · Models & Laws · Empirical Methods

Course: HCI 06.103.115 (IIT Guwahati · NPTEL 106103115) · Instructors: Dr. Samit Bhattacharya, Prof. Pradeep Yammiyavar

CONTENTS

PART I

Guidelines for HCI

1. Shneiderman's Eight Golden Rules

Who & where: Ben Shneiderman, *Designing the User Interface* (1987, currently 6th ed. with Plaisant). High-level design rules for interactive systems.

Mnemonic: *"Consistent feedback prevents errors; undo gives control; memory is short."*

#	Rule	Meaning	Quick example
1	**Strive for consistency**	Same actions in same situations; same terminology in menus, prompts, help.	"Submit" is identical on every page; `Ctrl-S` saves everywhere.
2	**Seek universal usability**	Cater to novices and experts, kids and elderly, abled and disabled.	Beginner mode + keyboard shortcuts for experts.
3	**Offer informative feedback**	Every action gets a system response — visible, timely, proportional.	Spinner during loading; green tick after save.
4	**Design dialogs to yield closure**	Action sequences have a clear beginning, middle, *end*.	Checkout: cart → address → pay → **"Order confirmed!"**
5	**Prevent errors**	Make errors hard; if they happen, give recovery.	Grey out invalid options; require typing "DELETE" to confirm.
6	**Permit easy reversal of actions**	Undo / back / cancel everywhere — reduces anxiety.	Cmd-Z; "Unsend" in Gmail; "Cancel order in 60s" in Swiggy.
7	**Keep users in control** (internal locus)	Users initiate; system responds — never the reverse.	Don't auto-reload the page; don't force-close their work.
8	**Reduce short-term memory load**	Don't make users remember things across screens.	"Step 2 of 4"; autofill saved addresses.

> EXAM TIP
>
> Distinguish **5 (Prevent errors)** from **6 (Reversal)**. Prevention is *upstream* — design the error out; reversal is *downstream* — let the user fix it.

2. Norman's Seven Design Principles

Who & where: Donald Norman, *The Design of Everyday Things* (1988; rev. 2013, chapter 7).

Do NOT confuse these with Norman's 7 stages of action (next section). The 7 principles are *how to design well*; the 7 stages describe *how users think*.

#	Principle	Plain meaning	Example
1	**Use both knowledge in the world AND in the head**	Don't make people memorise; put cues in the interface.	An ATM shows the card slot (world). You only remember your PIN (head).
2	**Simplify the structure of tasks**	Break complex tasks into small steps; use technology to remove mental load.	Microwave: 4 buttons (popcorn, reheat, pizza, defrost) instead of typing exact seconds.
3	**Make things visible — bridge the two gulfs**	Show what's possible (execution) and what just happened (evaluation).	A volume knob that turns *and* shows the current level.
4	**Get the mappings right**	Control layout matches the layout of what it controls.	Stove knobs arranged 2×2 to match a 2×2 burner grid.
5	**Exploit the power of constraints**	Make the wrong action impossible.	USB-C reversible; SIM-card slot only fits one way.
6	**Design for error**	Assume errors will happen; make them recoverable.	Trash folder (deleted files aren't gone for 30 days).
7	**When all else fails, standardize**	If you can't make it intuitive, make it the same everywhere.	Red = stop globally; iOS back button on the top-left.

Memory trick: `KSV-MCE-S` → Knowledge · Simplify · Visibility · Mapping · Constraints · Error · Standardize.

3. Norman's Model of Interaction — Seven Stages of Action

This is **how the user thinks** when interacting. It is a cognitive model — descriptive, not prescriptive.

The Seven Stages

```
           ┌─ 1. Forming the GOAL ─────────────────────┐
           │   (what I want: "make coffee")             │
           │                                            ▼
           │   EXECUTION SIDE                     EVALUATION SIDE
    2. Forming the INTENTION              7. EVALUATING the outcome
       ("I'll use the kettle")               ("did I get coffee?")
           │                                        ▲
           ▼                                        │
    3. Specifying the ACTION              6. INTERPRETING perception
       ("press kettle switch")               ("light on → must be heating")
           │                                        ▲
           ▼                                        │
    4. EXECUTING the action ────────────→ 5. PERCEIVING the state
       (finger pushes button)                 (I see the indicator light)
                                                   ↑
                                     the WORLD changes here
```

The Two Gulfs — most-tested concept

Gulf	Where	What it is	Designer's job
Gulf of Execution	Between **intention** and **action** (stages 2 → 4)	Gap between *what the user wants to do* and *what the system lets them do.*	Make actions visible, easy, obvious — **affordances + signifiers**.
Gulf of Evaluation	Between **perception** and **goal** (stages 5 → 7)	Gap between *what the system shows* and *what the user can understand* from it.	Give clear, timely **feedback** mapped to the user's goal.

Concrete example — Setting an alarm clock

1. **Goal:** "Wake up at 6 am."
2. **Intention:** "Set the alarm to 6 am."
3. **Specify action:** "Press 'alarm-set', then arrows to 6:00."
4. **Execute:** Press the buttons.
5. **Perceive:** See the display change to "6:00 ⏰ ".
6. **Interpret:** "OK, screen says 6:00 with an alarm icon — alarm armed."

7. **Evaluate:** "Yes — goal achieved."

A bad alarm clock makes step 3 painful (Gulf of Execution: which button does what?) *or* step 6 painful (Gulf of Evaluation: is the alarm armed or not?).

> EXAM TIP
>
> Pair each gulf with the Norman principle that fixes it: **Execution** → good mappings + affordances + constraints. **Evaluation** → visibility + feedback.

4. Nielsen's Ten Usability Heuristics

Who: Jakob Nielsen (with Rolf Molich originally), revised 1994. Rules of thumb for *evaluating*, not building.

H1. Visibility of system status

The system should always keep users informed about what is going on, through appropriate feedback within reasonable time.

- **Good:** File upload shows a progress bar with "12 MB / 30 MB · 2 min left".
- **Bad:** Click "Save" → nothing happens for 6 seconds → user clicks again → duplicate save.

H2. Match between system and the real world

Speak the user's language; follow real-world conventions.

- **Good:** "Move to trash" (real-world metaphor).
- **Bad:** Error message: `ERR_RFC2822_LOCAL_PART_INVALID`.

H3. User control and freedom

Provide a clearly marked emergency exit; support undo and redo.

- **Good:** Cmd-Z; Gmail's "Undo send" timer.
- **Bad:** A 5-step wizard with no Back button.

H4. Consistency and standards

Users shouldn't wonder whether different words, situations or actions mean the same thing.

- **Good:** All save buttons say "Save" in the same place, same colour.
- **Bad:** "Submit" → "Send" → "Confirm" for the same action across three pages.

H5. Error prevention

Better than good error messages is a design that prevents the problem.

- **Good:** Date picker (you cannot type Feb 30).
- **Bad:** Delete-account button next to Save, same colour, no confirmation.

H6. Recognition rather than recall

Make objects, actions and options visible. Don't burden working memory.

- **Good:** Autocomplete dropdown showing recent searches.
- **Bad:** A CLI where users must remember every flag (`-rstv`).

H7. Flexibility and efficiency of use

Accelerators — invisible to novices — speed up experts.

- **Good:** `Cmd-K` command palette in VS Code; every action also in menus.
- **Bad:** Every action requires 4 nested menu clicks. No shortcuts.

H8. Aesthetic and minimalist design

Dialogues should not contain irrelevant or rarely-needed information.

- **Good:** Dashboard with 3 hero metrics; rest behind "More ▾ ".
- **Bad:** 38 metrics on one screen.

H9. Help users recognize, diagnose, and recover from errors

Plain language, identify the problem, suggest a solution.

- **Good:** "Password must be at least 8 characters. You entered 5." (with field highlighted)
- **Bad:** Red banner saying just "Form invalid."

H10. Help and documentation

Even better if no docs needed; provide them anyway, searchable.

- **Good:** Inline tooltip: "What's an IFSC? It's an 11-character code, top-right of your cheque."
- **Bad:** "?" icon opens a 200-page PDF.

> EXAM TIP
>
> Examiners love asking you to identify *which* heuristic is violated by a given scenario. Practise: read 5 violation scenarios, label each with H1–H10. Most-confused pair: **H1 (visibility)** = ongoing status, **H9 (error recovery)** = after a failure.

5. Heuristic Evaluation – the Method

Who: Nielsen & Molich (1990). A *discount usability inspection method* — cheap, fast, no users needed.

Procedure (5 steps)

1. **Brief the evaluators** — explain the system and task scenarios.
2. **Independent evaluation** — each evaluator inspects the interface *alone*, 2–3 passes, listing violations of the 10 heuristics. (Independence prevents groupthink.)
3. **Severity rating** — each violation gets a 0–4 rating.
4. **Aggregation** — combine all lists; remove duplicates; produce a master severity-ranked list.
5. **Debrief** — group discussion (often with developers) to interpret findings and brainstorm fixes.

Nielsen's Severity Scale

Rating	Meaning
0	Not a usability problem at all.
1	**Cosmetic** — fix only if extra time available.
2	**Minor** — fixing has low priority.
3	**Major** — fixing has high priority.
4	**Catastrophe** — must fix before release.

Empirical finding (often asked)

- **1 evaluator** finds ~35% of problems.
- **3–5 evaluators** find ~75% of problems.
- **10 evaluators** find ~90%, with diminishing returns.
- → **3–5 evaluators is the cost-effective sweet spot.**

Strengths and weaknesses

Strengths	Weaknesses
Cheap (no users, no lab).	Misses problems specific to real users.
Fast (a day or two).	Evaluator expertise matters a lot.
Catches many issues early.	Generates many false positives.
Works on mockups, not just live systems.	Misses domain-specific issues.

6. Contextual Inquiry — the Field-Research Method

Who: Hugh Beyer & Karen Holtzblatt, *Contextual Design* (1998). Roots in ethnography.

What it is

A semi-structured **interview that happens in the user's actual work environment, while they do their actual work**. Not in a lab, not retrospective.

The master / apprentice model

The user is the **master** (they know the work). The researcher is the **apprentice** (learning from them). The researcher watches, asks "why are you doing that?", and shares interpretations back for the user to confirm or correct.

Beyer & Holtzblatt's Four Principles

#	Principle	Meaning
1	**Context**	Go to where the work is done. Real workplace, real tools, real distractions.
2	**Partnership**	Researcher and user are collaborators, not interviewer/subject.
3	**Interpretation**	Researcher shares interpretations *during* the inquiry; user confirms or corrects.
4	**Focus**	Researcher steers conversation toward the project's goals without forcing it.

Concrete example

Designing a hospital intake system. Instead of interviewing nurses in a meeting room, you sit beside a nurse on her morning shift. You watch her register 12 patients, ask *"why did you write* *that on a sticky note and not the system?"*, and discover that the system's keyboard layout doesn't match a wet-hand workflow. **You'd never have learned this from an interview.**

When to use CI

- Early in design — to understand the problem domain.
- When the work is complex or socially embedded.
- When users can't articulate what they do (most tacit work).

Strengths vs weaknesses

Strengths	Weaknesses
Rich, situated, real-world data.	Time-intensive (hours per user).
Surfaces tacit / unconscious behaviour.	Hard to scale; small sample sizes.
Builds empathy in the design team.	Researchers must be trained in observation.

7. Cognitive Walkthrough – the Task-Focused Inspection Method

Who: Polson, Lewis, Rieman, Wharton (early 1990s). Designed to evaluate **learnability** — how easy it is for a *novice* who has never seen the system before.

Core idea

The evaluator pretends to be a novice user with a specific goal, and *walks step by step* through the action sequence the designer intended. At each step, the evaluator asks four questions.

The Four Questions (memorise verbatim)

1. **Will the user try to achieve the right effect?** (Does the user understand that the current sub-goal is the right next step?)
2. **Will the user notice that the correct action is available?** (Is the button or control visible and identifiable?)
3. **Will the user associate the correct action with the effect they are trying to achieve?** (Is the label/icon clear enough?)
4. **If the correct action is performed, will the user see that progress is being made toward solution?** (Does the system give reassuring feedback?)

Procedure

1. Define inputs: who the user is, what task they're doing, the correct action sequence, an interface description.
2. For each action in the sequence, answer all four questions.
3. Document failures: any "no" answer is a usability problem — note it and theorise *why* the user would fail there.
4. Aggregate findings and produce a severity-ranked report.

Worked example – "Send a photo via WhatsApp Web"

Step	Action	Q1 right effect?	Q2 see action?	Q3 associate?	Q4 feedback?
1	Open chat	✓	✓ list clear	✓	✓ chat opens
2	Click attach	✓	⚠ small paperclip	✓ universal icon	✓ menu opens
3	Choose "Photos & Videos"	✓	✓	✓	✓ file picker

A ⚠ or ✗ at any step = a usability problem.

When to use

- **Early** in design (on mockups, even paper).
- When **learnability** is the priority — one-time-use kiosks, public services, onboarding flows.

Methods compared — the table that always appears in exams

Aspect	Heuristic Evaluation	Cognitive Walkthrough	Contextual Inquiry
Type	Expert inspection	Expert inspection	User research (field)
Needs real users?	No	No (experts simulate)	**Yes** (in their workplace)
Focus	Whole interface, holistic	One specific task, step by step	The work itself, in context
Measures	General usability vs heuristics	**Learnability** (first-time use)	Tacit work practices, needs
Best stage	Anytime (mockup → live)	Early (mockups)	Earliest (problem discovery)
Number of evaluators/users	3–5 evaluators	1–3 evaluators	4–8 users (rich detail each)
Cost	Low	Low	High (hours per session)
Strength	Cheap, fast, broad coverage.	Targeted, catches missed cues.	Discovers what you didn't know to ask.
Weakness	Generates false positives.	Narrow (one task at a time).	Time-consuming, small N.

Two-minute Part-I revision

- **Shneiderman 8:** Consistency · Universal · Feedback · Closure · Prevent · Reverse · Control · Memory.
- **Norman's 7 principles:** Knowledge · Simplify · Visibility · Mapping · Constraints · Error · Standardize.
- **Norman's 7 stages:** Goal → Intention → Action-spec → Execute → Perceive → Interpret → Evaluate. Two gulfs: **Execution** (intend → do) and **Evaluation** (see → understand).
- **Nielsen 10:** Visibility · Match · Control · Consistency · Prevention · Recognition · Flexibility · Aesthetic · Recovery · Help.
- **Heuristic eval:** 3–5 expert inspectors, severity 0–4, finds ~75% of issues.
- **Contextual inquiry:** field study, master/apprentice, four principles (Context · Partnership · Interpretation · Focus).
- **Cognitive walkthrough:** task-step-by-step, 4 questions, focus on learnability.

PART II

Models · Fitts & Hick · Case Studies

8. Different Types of Models in HCI

8.1 Classification by purpose

Type	What it does	Example
Descriptive	*Describes* behaviour — explains how things work.	Norman's 7 stages of action.
Predictive	*Predicts* a quantity before you build.	Fitts's Law, KLM, Hick-Hyman.
Prescriptive	*Tells you what to do.*	Nielsen's heuristics, Shneiderman's golden rules.

8.2 Classification by what is modelled

Type	Models the...	Example
User models	*user's* knowledge, abilities, preferences.	MHP, ACT-R, personas.
Task models	*task* the user does.	HTA, GOMS, CTT.
System/dialogue models	*system's* states and transitions.	State Transition Networks, statecharts.

8.3 The Model Human Processor (MHP)

Who: Card, Moran & Newell (1983). The foundational user model in HCI.

```
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│  PERCEPTUAL      │   │    COGNITIVE     │   │      MOTOR       │
│  PROCESSOR       │───│    PROCESSOR     │───│    PROCESSOR     │
│  (eyes, ears)    │   │  (working mem.)  │   │  (hands, voice)  │
│  τp ≈ 100 ms     │   │   τc ≈ 70 ms     │   │   τm ≈ 70 ms     │
└──────────────────┘   └──────────────────┘   └──────────────────┘
                                │
                                ▼
                     Long-Term Memory (~unlimited)
                     Working Memory (~7 chunks, ~7s decay)
```

Processor	Symbol	Cycle time	Range
Perceptual	τp	**100 ms**	50–200 ms
Cognitive	τc	**70 ms**	25–170 ms
Motor	τm	**70 ms**	30–100 ms

Typical MHP question: How long to see an alert, decide it requires action, press OK? Answer: $\tau_p + \tau_c + \tau_m \approx 100 + 70 + 70 =$ **240 ms minimum**.

8.4 GOMS family

GOMS = **Goals · Operators · Methods · Selection rules.** A task model that decomposes how an expert (error-free) user accomplishes a goal.

Variant	Distinguishing feature
CMN-GOMS (original)	Strict goal-method hierarchy; serial operators.
KLM (Keystroke-Level)	Simplest — only 5 operators (K, P, H, M, R); single method.
NGOMSL	Adds explicit notation; predicts *learning* time too.
CPM-GOMS	Allows **parallel** operations (MHP scheduling); for skilled tasks like phone-operator work.

Op	Action	Time (s)
K	Keystroke	0.20 (average typist)
P	Pointing	1.10
H	Homing (kbd ↔ mouse)	0.40
M	Mental preparation	1.35
R	System response	variable

Rules for M operators: place an M before each task chunk; remove an M when the next step is fully anticipated (typing a known sequence); remove M if the user is already mentally in the task.

8.5 Hierarchical Task Analysis (HTA)

Who: Annett & Duncan (1967). A descriptive task model: a tree decomposing a goal into sub-goals until each leaf is an observable action. Each parent has a **plan** describing order.

```
0. Make tea
   1. Boil water
      1.1 Fill kettle
      1.2 Switch kettle on
      1.3 Wait for kettle
   2. Prepare cup
      2.1 Put tea bag in cup
      2.2 (Optionally) add sugar
   3. Pour water into cup
   4. Wait 3 minutes
   5. Remove bag, add milk
   Plan 0: do 1 and 2 in any order, then 3, then 4, then 5.
   Plan 1: 1.1 → 1.2 → 1.3.
```

8.6 State Transition Networks (STN)

System/dialogue model. Nodes = states; arrows = user-triggered transitions; labels = actions. Used to find **unreachable** states and **dead-end** states (H3 violations).

```
   [IDLE] ──insert card──▶ [WAIT_PIN] ──valid PIN──▶ [MENU]
                               │                        │
                           invalid×3                    │
                               ▼                amount + confirm
                           [BLOCKED]                    │
                                                        ▼
                                                [DISPENSE] ──done──▶ [IDLE]
```

8.7 Classification practice

Model	Descriptive/Predictive/Prescriptive	User/Task/System
MHP	Descriptive (of the user)	User
HTA	Descriptive	Task
KLM	**Predictive**	Task (+ user assumed)
Fitts's Law	**Predictive**	User (motor)
Nielsen's heuristics	**Prescriptive**	(meta — none)
STN	Descriptive	System

9. Fitts's Law – In Depth

Who: Paul Fitts (1954), *The information capacity of the human motor system in controlling the amplitude of movement.*

The formula

Shannon–Welford form (modern, used in ISO 9241-9):

$$MT = a + b \cdot \log_2(D/W + 1)$$

Original Fitts form:

$$MT = a + b \cdot \log_2(2D/W)$$

- **MT** = movement time (seconds)
- **D** = distance to centre of target
- **W** = target width along direction of movement
- **a, b** = empirical constants for the device + user. Typical mouse values: $a \approx 0.10$ s, $b \approx 0.15$ s/bit.

Index of Difficulty & Throughput

$$ID = \log_2(D/W + 1) \quad \text{bits}$$

$$TP = ID \,/\, MT \quad \text{bits/sec}$$

Mouse throughput is typically **3.5–4.5 bits/sec**. Touch is similar. Eye-gaze is ~1 bit/sec.

Worked numerical problems

Example 1 – Simple MT

Button 20 px wide, 200 px away. a = 0.10 s, b = 0.15 s/bit.

```
ID = log₂(200/20 + 1) = log₂(11) ≈ 3.459 bits
MT = 0.10 + 0.15 × 3.459 ≈ 0.619 s
```

Example 2 – Comparing two designs

A: W=40, D=100. B: W=80, D=100. Which is faster?

```
ID_A = log₂(100/40 + 1) = log₂(3.5) ≈ 1.807 bits → MT ≈ 0.371 s
ID_B = log₂(100/80 + 1) = log₂(2.25) ≈ 1.170 bits → MT ≈ 0.275 s
Design B faster by ≈ 96 ms per click.
```

Example 3 – Throughput

User clicks targets with ID = 4 bits at 0.7 s avg. TP = 4 / 0.7 ≈ **5.71 bits/sec**.

Variants & extensions

- **Welford's version** — accounts for asymmetry of close vs far movements.
- **Accot–Zhai** — for *trajectory* movement (steering through a tunnel): MT = a + b·(A/W).
- **2D Fitts** — uses the smaller of width and height when the target is rectangular and the approach is not axis-aligned.

Design implications

1. Make important controls big.
2. Place them close to where the cursor already is.
3. Edges and corners are effectively infinite (cursor stops there) — macOS menu bar at the very top.
4. Pie/radial menus: equal ID for each option, faster than linear menus.
5. Pop-up sub-menus open *toward* the current cursor.

Limitations

- Assumes pointing is the bottleneck (ignores decision time — use Hick).
- Doesn't account for fatigue, visual occlusion, or context switching.
- Empirical constants vary across users and devices.

10. Hick–Hyman Law – In Depth

Who: William Hick (1952), Ray Hyman (1953). Decision time for one choice from N alternatives.

The formula

Equiprobable choices:

$$RT = a + b \cdot \log_2(n + 1)$$

Non-equiprobable (entropy-weighted, original Hyman):

$$RT = a + b \cdot H, \quad H = \Sigma\, p_i \cdot \log_2(1/p_i)$$

H is the **information entropy** of the choice set (bits). When all options are equiprobable, H reduces to $\log_2(n)$; adding the +1 gives $\log_2(n+1)$.

Worked examples

Example 1 – Simple menu

5 equally-likely items. a = 0.20 s, b = 0.25 s/bit.

```
RT = 0.20 + 0.25 × log₂(6)
   = 0.20 + 0.25 × 2.585
   ≈ 0.846 s
```

Example 2 – Flat vs hierarchical menus

A: flat 16-item menu. B: 4 menus of 4 (two-step).

```
RT_A = a + b · log₂(17) ≈ 0.2 + 4.087 × 0.25 = 1.22 s
RT_B = 2 × (a + b · log₂(5)) = 2 × (0.2 + 2.322 × 0.25) = 1.56 s
```

Counter-intuitive: **flat is faster for a one-shot choice** — but for learning and visual scanning, hierarchical wins. Real menus also have Fitts costs.

Example 3 – Non-equiprobable choices

OS file menu: Save 70%, Open 20%, Print 7%, Quit 3%.

```
H = 0.7 log₂(1/0.7) + 0.2 log₂(1/0.2)
  + 0.07 log₂(1/0.07) + 0.03 log₂(1/0.03)
H ≈ 0.36 + 0.46 + 0.27 + 0.15 = 1.24 bits
```

Equiprobable 4-option $H = \log_2(4) = 2$ bits. A skewed distribution makes decisions faster — which is why most-used commands should be most prominent.

Design implications

1. Don't show 100 options at once. Chunk, hide, tier.
2. Most-frequent first — sort by usage frequency to drop entropy.
3. Limit choices in time-critical UI (emergency: ≤ 3 buttons).
4. Search vs browse — search bypasses Hick entirely when n is huge.

Limitations

- Assumes the user has *learned* the choices.
- Breaks down for very large n (> ~20 — different cognitive process).
- Equiprobable formula ignores recency, familiarity, prominence.

Fitts vs Hick–Hyman — the comparison

Aspect	Fitts's Law	Hick–Hyman's Law
Models what	**Movement** time (motor)	**Decision** time (cognitive)
Inputs	Distance D, Width W	Number of choices n
Formula	$MT = a + b \cdot \log_2(D/W + 1)$	$RT = a + b \cdot \log_2(n + 1)$
Index	ID (Index of Difficulty)	H (Information entropy)
When it dominates	Reaching for a target	Choosing among many targets
Designer's lever	Size and position of controls	Number and grouping of controls
MHP processor	Motor	Cognitive
Improves with practice?	Mildly (constants shift)	**Strongly** (familiar = automatic)

TRICK QUESTION

"A user clicks the largest of 16 items on a screen. Which law dominates?" — **Both**. Hick for deciding which to click; Fitts for moving to it. Total ≈ RT (Hick) + MT (Fitts).

11. Model-Based Design – Three Case Studies

11.1 Case 1 – Banking app transfer flow (KLM)

Problem: Public-sector banking app for first-time smartphone users. ~60% abandonment on the 7-field transfer form.

Step 1 – HTA of existing flow

```
0. Send money to a payee
   1. Open Quick Transfer
   2. Fill form
      2.1 Select source account
      2.2 Type payee account number
      2.3 Type payee name
      2.4 Type IFSC code
      2.5 Type amount
      2.6 Type remarks
      2.7 Select purpose code
   3. Confirm
   4. Authenticate (MPIN)
```

Step 2 – KLM analysis of "₹500 transfer"

Step	Operators	Time
Open app	M + P + K	2.65 s
Find "Quick Transfer"	M + P + K	2.65 s
Tap account	P + K	1.30 s
Type 16-digit account	M + 16K	4.55 s
Tap name field	P + K	1.30 s
Type name (8 chars)	8K	1.60 s
Type IFSC	M + 11K	3.55 s
Type amount	M + 3K	1.95 s
Tap continue	P + K	1.30 s
Confirm	P + K	1.30 s
Enter MPIN	M + 4K	2.15 s
Total		**≈ 24.3 s**

Step 3 – Redesign

- **Fitts:** 96 px primary tap target (bottom third of screen).
- **Hick:** one-question-per-screen.
- **Recognition (H6):** "Send again" surfaces last 3 payees as one-tap chips.

Step 4 – KLM of redesigned flow

Step	Time
Open app	2.65 s
Tap "Send again" tile	1.30 s
Confirm amount (₹500 default)	1.30 s
MPIN	2.15 s
Total	**≈ 7.4 s**

Result: 3.3× speedup, predicted *before* any user testing.

11.2 Case 2 – Rural hospital kiosk sizing (Fitts)

Problem: PHC Sahyog kiosk. Past pilot failed — elderly patients pecked at tiny buttons.

Constraints: Patient 1.4 m from screen; pointing tolerance ±3 cm (shaky hand, single finger).

Fitts calculation at W = 6 cm

```
ID = log₂(60/6 + 1) = log₂(11) = 3.46 bits
MT = 0.10 + 0.15 × 3.46 ≈ 0.62 s per tap
```

If shrunk to W = 1.5 cm

```
ID = log₂(60/1.5 + 1) = log₂(41) ≈ 5.36 bits
MT ≈ 0.90 s per tap, with much higher error rate
```

Decision: enforce ≥ 6 cm (96 px) targets. Hick limits categories to 4 (RT ≈ 0.78 s). Combined KLM: full registration ≈ 7.4 s vs the paper-desk 9–14 min.

11.3 Case 3 – NPTEL dashboard IA (HTA + Hick)

Problem: 14 top-level tabs, 3 carousels, 2 banners. Students can't find anything.

HTA of "submit this week's assignment" (old)

```
0. Submit assignment
   1. Find assignment
      1.1 Open dashboard
      1.2 Scan 14 tabs → "My Courses"
      1.3 Click "My Courses"
      1.4 Scroll past 4 banners
      1.5 Open HCI course card
      1.6 Find "Assignments" tab
      1.7 Open Week 6
   2. Upload file
   3. Confirm
```

Hick at "find" steps

```
Step 1.2: H = log₂(14 + 1) ≈ 3.91 bits → RT ≈ 1.18 s
Step 1.6: H = log₂(7 + 1)  = 3.00 bits → RT ≈ 0.95 s
Total decision time across 7 sub-steps ≈ 6.1 s
```

Card sort with 18 students

Collapses 34 dashboard objects into 3 buckets: *due now*, *keep learning*, *explore later*.

Redesigned HTA

```
0. Submit assignment
   1. Open dashboard
   2. Tap "Assignment 6 — in 2d 14h" (Due Now card)
   3. Upload + confirm
```

Decision time per step now: $H = \log_2(3 + 1) = 2$ bits → RT ≈ 0.70 s. Total decision time ≈ 1.4 s vs 6.1 s. Full KLM: ~33 s → ~9 s.

The Models Map

```
                 ┌─────────────────────────────┐
                 │   MODEL HUMAN PROCESSOR     │
                 │ τp=100ms · τc=70ms · τm=70ms │
                 └──────────────┬──────────────┘
                                │ provides cycle times to:
               ┌────────────────┼────────────────┐
               ▼                ▼                ▼
         ┌────────────┐   ┌────────────┐   ┌────────────┐
         │  Fitts's   │   │   GOMS /   │   │   Hick-    │
         │    Law     │   │    KLM     │   │   Hyman    │
         │  (motor)   │   │(task time) │   │(cognitive) │
         └─────┬──────┘   └─────┬──────┘   └─────┬──────┘
               │                │                │
               └────────┬───────┴────────┬───────┘
                        ▼                ▼
                ┌──────────────┐   ┌──────────────┐
                │     HTA      │   │     STN      │
                │ (task tree)  │   │ (system FSM) │
                └──────┬───────┘   └──────┬───────┘
                       └─────────┬────────┘
                                 ▼
                 ┌─────────────────────────┐
                 │   MODEL-BASED DESIGN    │
                 │ (predict → redesign →   │
                 │  re-predict → ship)     │
                 └─────────────────────────┘
```

Part-II revision summary

- **MHP:** three processors P/C/M, cycle times 100/70/70 ms.
- **GOMS/KLM:** 5 operators K, P, H, M, R; predicts expert task time.
- **HTA:** hierarchical decomposition of a task; descriptive.
- **STN:** finite-state machine for system dialogue.

- **Fitts:** $MT = a + b \cdot \log_2(D/W + 1)$; ID in bits; throughput $TP = ID/MT$.
- **Hick:** $RT = a + b \cdot \log_2(n + 1)$; use entropy H for non-equiprobable.
- **Case studies pattern:** HTA → KLM/Fitts/Hick predictions → redesign → re-predict.

PART III

Empirical Research Methods

12. Motivation – Why Empirical Research in HCI?

The core question: *"Does this design actually work better than the alternative?"*

You **cannot** answer this by:

- Asking the designer (biased).
- Asking the user's opinion (poor predictor of behaviour).
- Intuition or expert review alone (catches ~75% of issues).
- Following rules (don't tell you *how much* difference they make).

You **can** answer it by measuring what users actually *do* in a controlled study, comparing conditions, and using statistics to decide whether the difference is real or noise.

Two roles for empirical research

Role	Question it answers	Example
Engineering / formative	Is design A better than design B for *this* product?	Should we use a button or a swipe gesture?
Scientific / summative	What is the general relationship between X and Y?	How does font size affect reading speed on small screens?

13. Issues – Why HCI Experiments are Hard

1. **Human variability.** Two users doing the same task produce different times, error rates, preferences. Variance often exceeds the design effect. *Fix:* replication + within-subject designs + statistics.
2. **Learning effects (carry-over).** A user who has done task A is already practised by task B. *Fix:* counterbalancing, Latin squares.
3. **Fatigue effects.** Long sessions make users slower and more error-prone. *Fix:* ≤ 60 min sessions, breaks, counterbalancing.
4. **Ecological validity.** Lab ≠ real world. *Fix:* field studies, "in-the-wild" deployments, supplement with CI.

5. **Sampling bias.** Undergraduates with good vision aren't your real users. *Fix:* representative recruitment; report demographics.
6. **Hawthorne / observer effect.** People perform differently when watched. *Fix:* longitudinal designs, acknowledged limitations.
7. **Ethical constraints.** Informed consent; no deception without IRB; anonymise data.
8. **Cost & time.** 30-participant × 4-condition study ≈ 80 researcher hours. *Fix:* pilots, online panels (Prolific), unmoderated tools.

14. Research Question Formulation

14.1 SMART criteria

Letter	Criterion	Bad	Good
S	Specific	"Are dropdowns good?"	"Does a single-column dropdown produce faster selection on a 6-inch smartphone than a multi-column grid?"
M	Measurable	"Do users like X better?"	"Do users rate X higher on a 7-point Likert?"
A	Achievable	"What is the optimal interface?"	"Among three candidate layouts, which yields the lowest error rate?"
R	Relevant	"Are red buttons faster than blue?" (irrelevant)	"For colour-blind users, do colour-coded warnings reduce false dismissals?"
T	Time-bound	"Eventually we should test it."	"One 45-min session per user with 30 users."

14.2 FINER criteria

- **F**easible · can it be done with available resources?
- **I**nteresting · will the answer matter?
- **N**ovel · already answered? (Replication fine but say so.)
- **E**thical · no harm, no deception.
- **R**elevant · advances theory or practice.

14.3 PICO(T) framework

Letter	Stands for	Example
P	Population	First-time smartphone users aged 50+.
I	Intervention	One-question-per-screen banking form.
C	Comparison	Existing multi-field form.
O	Outcome	Transfer success rate & time-to-completion.
T	Time	Single 30-minute session.

14.4 From question to hypothesis

- **H_0 (null):** No difference between conditions.
- **H_1 (alternative):** There IS a difference.
- **One-tailed** = directional. **Two-tailed** = either direction.

15. Experiment Design

15.1 Variables

Variable	Definition	Example
Independent (IV)	What the experimenter manipulates.	Button size (small/medium/large).
Dependent (DV)	What the experimenter measures.	Click time, error rate, satisfaction.
Control	Held constant to prevent interference.	Brightness, room temperature, time of day.
Confounding	An *uncontrolled* variable that varies with IV.	Testing blue vs red but blue always shown first.

15.2 Factors, levels, conditions

- **Factor** = single IV.
- **Level** = one value of a factor (small/medium/large).
- **Condition** = unique combination of levels across all factors.

Two factors — button size (3 levels) × shape (2 levels) — gives a **3 × 2 factorial design** with 6 conditions.

15.3 Between vs Within vs Mixed

Design	Pros	Cons
Between-subjects (each user → one condition)	No order effects · simpler instructions.	More participants · individual-difference noise · less power.
Within-subjects (each user → all conditions)	Fewer participants · each user is own control · more power.	Order effects (learning, fatigue) · longer sessions.
Mixed	Some factors between, others within.	More complex analysis.

Default: use within-subjects with counterbalancing. Switch to between when a condition permanently changes the user (training, skill acquisition).

15.4 Counterbalancing & Latin Squares

In within-subject designs with k conditions, vary the order so order doesn't confound condition.

Full counterbalancing uses all k! orders — quickly impractical (4! = 24 orders).

Latin square: a k×k arrangement where each condition appears exactly once in each row and once in each column. Example with 4 conditions A B C D:

```
            Pos 1   Pos 2   Pos 3   Pos 4
Group 1:      A       B       C       D
Group 2:      B       C       D       A
Group 3:      C       D       A       B
Group 4:      D       A       B       C
```

Needs participants in multiples of 4. **Balanced Latin square:** a special form where each condition is preceded by every other equally often (use whenever k is even).

15.5 Sample size

- Pilot: 3–5 users.
- Heuristic eval: 3–5 evaluators.
- User testing (qualitative): 5–8 per persona.
- Quantitative comparison: 20–30 per condition.
- A/B test (production): thousands to millions.

Formal size is fixed by **power analysis** — effect size + α + desired power (≥ 0.80). Small effect ($d = 0.2$) needs ~200/group; large effect ($d = 0.8$) needs ~26/group.

15.6 Types of measurements (DV categories)

Type	Measures	Examples
Performance / behavioural	What users do.	Time-on-task, errors, success rate.
Subjective / preference	What users think.	Likert, SUS, NPS.
Physiological	Body response.	Eye-tracking, GSR, EEG.
Process	How they did it.	Click paths, retries, hesitations.

15.7 Validity types

Validity	Question	Threat
Internal	Did the IV cause the DV change?	Confounding variables.
External	Does it generalise beyond the study?	Unrepresentative sample, artificial setting.
Construct	Are we measuring what we think we are?	"Satisfaction" score that doesn't predict actual use.
Ecological	Does it apply to real-world contexts?	Lab ≠ in-the-wild.

16. Data Analysis

16.1 Two families

Descriptive — summarises data: mean, median, mode, SD, variance, range, quartiles, IQR, SEM.

Inferential — decides if differences are real.

- **α (significance level)** typically 0.05.
- **p-value:** probability of this data (or more extreme) given H_0 is true.
- $p < \alpha \rightarrow$ reject H_0.
- $p \geq \alpha \rightarrow$ fail to reject H_0 (NOT "prove no difference").
- **Type I error (α):** false positive.
- **Type II error (β):** false negative.
- **Power (1 – β):** probability of detecting a real effect; convention 0.80.

16.2 Choosing the right test

```
Is DV continuous (time, score) or categorical (success/fail)?

   CONTINUOUS                               CATEGORICAL
        |                                        |
   How many groups?                        Chi-square (χ²)
        |                                   (test of independence)
   ┌────┴─────┐
   |          |
  1 or 2     3+
   |          |
 t-test     ANOVA
              |
       post-hoc (Tukey HSD,
       Bonferroni) if significant
```

16.3 Common tests — when to use which

Test	Use when...	DV	Groups
One-sample t-test	Compare 1 group's mean to a known value.	continuous	1
Independent-samples t-test	Compare two *different* groups.	continuous	2 (between)
Paired-samples t-test	Compare *same* participants in 2 conditions.	continuous	2 (within)
One-way ANOVA	Compare 3+ groups on one factor.	continuous	3+ (between)
Repeated-measures ANOVA	Compare 3+ conditions within same participants.	continuous	3+ (within)
Two-way ANOVA	Two factors at once; tests interactions.	continuous	factorial
Chi-square (χ^2)	Categorical outcomes.	categorical	any
Mann–Whitney U	Non-parametric alternative to indep. t-test.	ordinal/skewed	2 (between)
Wilcoxon signed-rank	Non-parametric alternative to paired t-test.	ordinal/skewed	2 (within)
Kruskal–Wallis	Non-parametric alternative to one-way ANOVA.	ordinal/skewed	3+ (between)
Friedman	Non-parametric alternative to RM-ANOVA.	ordinal/skewed	3+ (within)

Use non-parametric tests when data is not normal, ordinal (Likert), small N (< 15/group), or has strong outliers.

16.4 Effect size – the OTHER number to report

Measure	Captures	Small / Medium / Large (Cohen)
Cohen's d	Standardised mean difference (two groups).	0.2 / 0.5 / 0.8
η^2 (eta-squared)	Proportion of variance explained (ANOVA).	0.01 / 0.06 / 0.14
r	Correlation strength.	0.1 / 0.3 / 0.5
Odds ratio	Categorical comparisons.	varies

Statistical significance ≠ practical importance. Always report both p and an effect size.

16.5 Worked example – paired-samples t-test

> **Problem.** 20 users perform the same task with two button designs (A and B).
> A: mean 12.4 s, SD 2.1 · B: mean 10.8 s, SD 1.9 · same participants for both.
> Is design B significantly faster?

1. **Design:** within-subjects, 2 conditions → **paired-samples t-test**.
2. **Hypotheses:** H_0: $\mu_A = \mu_B$; H_1: $\mu_A > \mu_B$.
3. **α** = 0.05.
4. **Test:** compute t = mean_diff / (SD_diff / √n); compare p to α.
5. **Effect size:** Cohen's d ≈ (12.4 − 10.8)/2.0 ≈ 0.8 → **large**.
6. **Interpretation:** "Design B reduced task time by 1.6 s on average (12.9% improvement). The difference was statistically significant ($p < 0.05$) with a large effect size (d ≈ 0.8). Design B is recommended."

17. Common Pitfalls

1. **p-hacking** — running many tests until one is significant. *Fix:* pre-register; Bonferroni correction.
2. **Cherry-picking** — reporting only "winning" conditions. *Fix:* report all tested.
3. **Correlation ≠ causation** — only randomised experiments support causal claims.
4. **Underpowered studies** — non-significant ≠ no effect when N is small.
5. **Ignoring effect size** — $p < 0.001$ with $d = 0.01$ is real but irrelevant.
6. **HARKing** — hypothesising after results are known. *Fix:* pre-registration.
7. **Ecological invalidity** — lab effect doesn't survive in the wild.

18. End-to-end Example

Scenario. Does a hold-to-confirm gesture reduce accidental deletions on a mobile notes app, vs simple tap-to-delete?

Step 1 – Research question (PICO)

"For young-adult users of a notes app (P), does a hold-to-confirm delete gesture (I) compared with a tap-to-delete (C) reduce accidental deletions (O) over a 20-minute session (T)?"

Step 2 – Hypotheses

- H_0: mean accidental deletions equal across conditions.
- H_1: hold-to-confirm yields lower mean accidental deletions.

Step 3 – Variables

- **IV:** delete mechanism (tap / hold).
- **DVs:** accidental-deletion count · task completion time · satisfaction (Likert 1–7).
- **Controls:** same notes, same phone, same script, neutral lighting.

Step 4 – Design

Within-subjects (each user tries both). Counterbalance order with Latin square.

Step 5 – Participants

30 users aged 18–24, university recruits, right-handed, normal vision (to control confounds).

Step 6 – Procedure

Informed consent → demographic survey → training trial → condition 1 (10 min) → break (2 min) → condition 2 (10 min) → satisfaction questionnaire → debrief.

Step 7 – Analysis

- Descriptive: mean ± SD per condition.
- Inferential: paired-samples t-test on accidental deletions and times (Wilcoxon if skewed).

- Report effect sizes (Cohen's d).

Step 8 – Sample result

"Hold-to-confirm reduced accidental deletions from $M = 4.2$ ($SD = 1.3$) to $M = 0.6$ ($SD = 0.7$), a statistically significant difference ($t(29) = 11.4$, $p < 0.001$, $d = 2.1$, very large effect). Task completion time increased slightly but non-significantly (10.4 s vs 10.9 s, $p = 0.18$). Satisfaction was equivalent (5.4 vs 5.5)."

Step 9 – Interpretation

Hold-to-confirm eliminates accidental deletions at negligible time cost and without harming satisfaction. **Recommendation: adopt hold-to-confirm for destructive actions.**

Part-III revision summary (last-night)

- **Motivation:** opinions are unreliable; data is necessary; magnitude only knowable empirically.
- **Issues:** variability, learning, fatigue, ecological validity, sampling bias, Hawthorne, ethics, cost.
- **RQ formulation:** SMART · FINER · PICO.
- **Variables:** IV / DV / control / confound.
- **Designs:** between · within · mixed; Latin square counterbalancing.
- **Validity:** internal · external · construct · ecological.
- **Hypothesis vocabulary:** H_0 / H_1 · one-tailed / two-tailed · Type I (α) / Type II (β) · $\alpha = 0.05$ · power = 0.80.
- **Test rule:** continuous + 2 groups → t-test; continuous + 3+ → ANOVA; categorical → χ^2; non-parametric versions for ordinal/skewed.
- **Always report:** descriptives + p-value + effect size (d, η^2, r).

Final-hour cheat sheet

PART I · GUIDELINES

Shneiderman 8: Consistency · Universal · Feedback · Closure · Prevent · Reverse · Control · Memory.
Norman 7 principles: Knowledge · Simplify · Visibility · Mapping · Constraints · Error · Standardize.
Norman 7 stages: Goal → Intention → Action-spec → Execute → Perceive → Interpret → Evaluate. Two gulfs (Execution + Evaluation).
Nielsen 10: Visibility · Match · Control · Consistency · Prevention · Recognition · Flexibility · Aesthetic · Recovery · Help.
Heuristic eval: 3–5 experts · severity 0–4 · finds ~75 %.
Contextual inquiry: field · master/apprentice · 4 principles (Context · Partnership · Interpretation · Focus).
Cognitive walkthrough: 4 questions · learnability focus.

PART II · MODELS & LAWS

MHP: τ_p 100 ms · τ_c 70 ms · τ_m 70 ms.
KLM ops: K 0.20 · P 1.10 · H 0.40 · M 1.35 · R ≈ 0.50.
Fitts: $MT = a + b \cdot \log_2(D/W + 1)$; ID in bits; $TP = ID/MT$.
Hick: $RT = a + b \cdot \log_2(n + 1)$; $H = \Sigma\, p \cdot \log_2(1/p)$ for non-equiprobable.
Case study pattern: HTA → KLM/Fitts/Hick → redesign → re-predict.

PART III · EMPIRICAL METHODS

RQ: SMART · FINER · PICO. **Hypothesis:** H_0 vs H_1 · one-/two-tailed.
Variables: IV · DV · control · confound.
Designs: between · within · mixed · Latin square.
Test rule: continuous + 2 → t-test · continuous + 3+ → ANOVA · categorical → χ^2.
Significance: $\alpha = 0.05$ · Type I = false +ve · Type II = false −ve · power ≥ 0.80.
Effect sizes: Cohen's d 0.2/0.5/0.8 · η^2 0.01/0.06/0.14.

HCI Study Guide · 06.103.115 · Spring 2026
Compiled for end-semester revision · all three theory blocks in one document.